

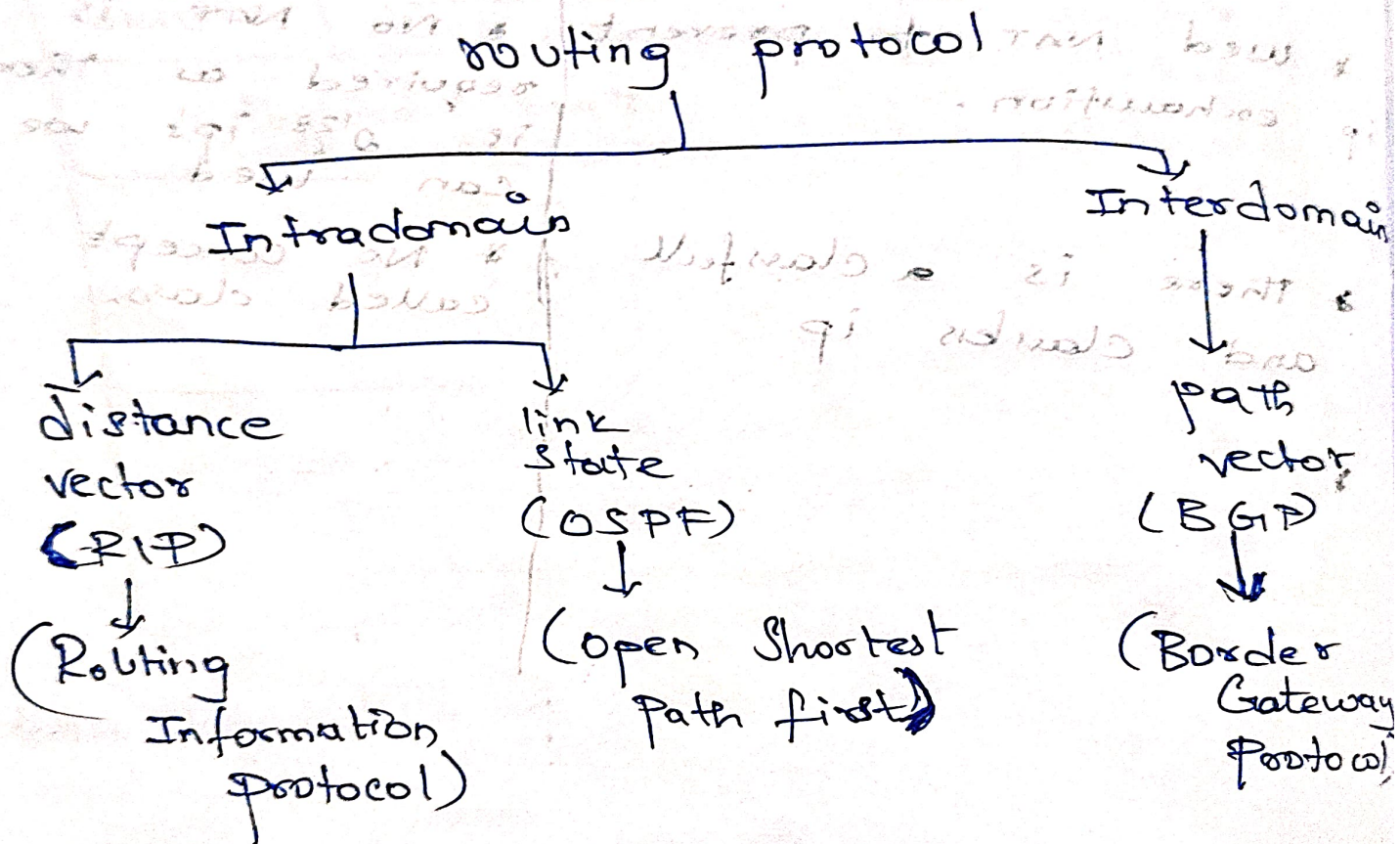
Unicast routing protocols

Autonomous System: Networks

* whatever is coming under a single Admin.

~~Any Autonomous~~

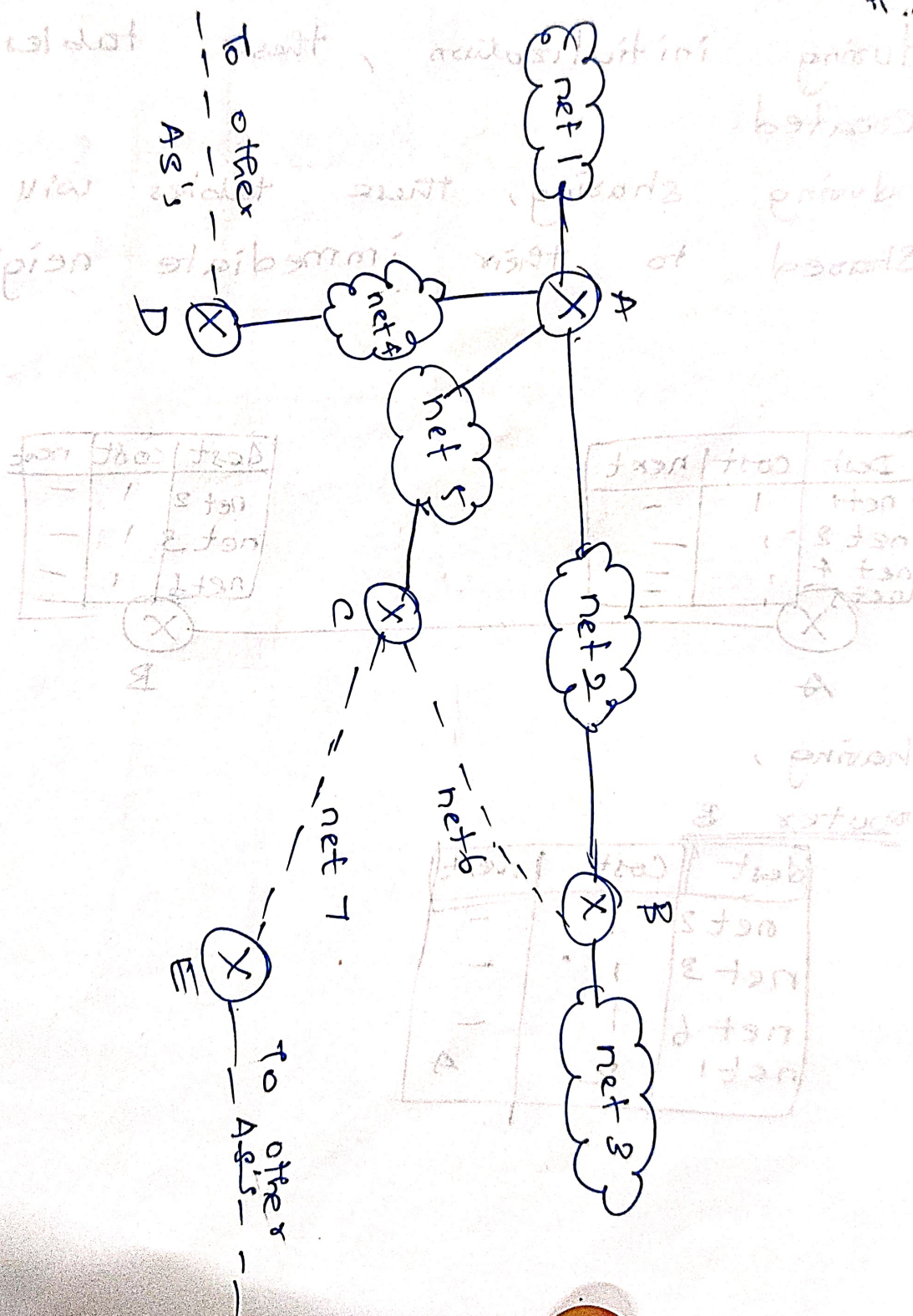
Any routing protocol working within any autonomous system, then the protocol is intra domain protocol



Distance Vector Routing:

- * Initialization
 - * Sharing
 - * Update
- phases

each node shares its routing table with its immediate neighbours periodically whenever it changes.



each router has a table with the directly connected network

Eg:
for R:

destination	Cost	Next hop
net 4	1	—

Network ip

during initialization, these tables are created.

during sharing, these tables will be shared to their immediate neighbours

Dest	cost	next
net 1	1	—
net 2	1	—
net 4	1	—
net 5	1	—

A

Dest	cost	next
net 2	1	—
net 3	1	—
net 6	1	—

B

Sharing,

Router B

dest	Cost	next
net 2	1	—
net 3	1	—
net 6	1	—
net 1	2	A

final routing table:

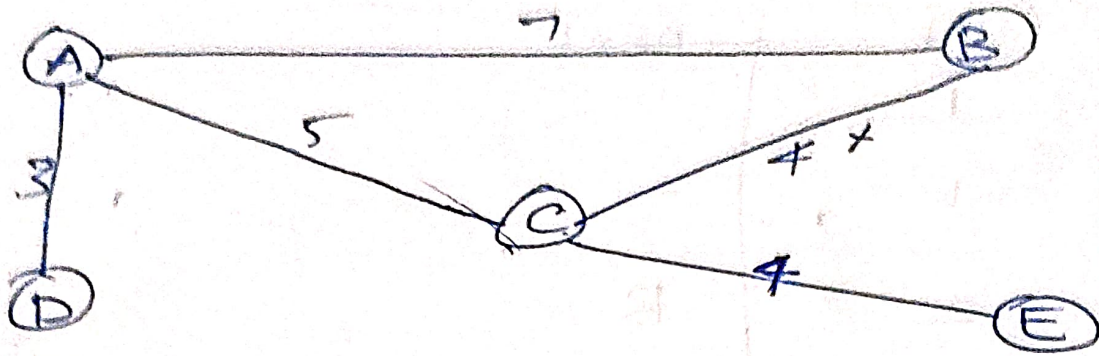
dest	cost	next
net 1	1	—
net 2	1	—
net 3	2	B
net 4	1	—
net 5	1	—
net 6	2	C
net 7	2	C

dest	cost	next
B	1	—
C	2	—
D	3	—
E	4	C

dest	cost	next
A	3	—
B	10	A
C	8	A
E	15	A

dest	cost	next
A	2	—
B	4	—
D	8	A
E	4	—

① Construct the routing table of each router.



router A

dest	cost	next
B	7	—
C	5	—
D	3	—
E	9	C

router B

dest	cost	next
A	7	—
C	4	—
D	10	A
E	8	C

router C

dest	cost	next
A	5	—
B	4	—
D	8	A
E	4	—

router D

dest	cost	next
A	3	—
B	10	A
C	8	A
E	12	A

dest	cost	next
A	9	C
B	8	C
C	4	-
D	12	C

reconstruct the routing table when there is a link failure between B & C.

router A:

dest	cost	next
B	7	-
C	5	-
D	3	-
E	9	C

router B:

dest	cost	next
A	7	-
C	12	A
D	10	A
E	16	A

router C

dest	cost	next
A	5	-
B	12	A
D	8	A
E	4	-

router D

dest	cost	next
A	3	-
B	10	A
C	8	A
E	12	A

router E

dest	cost	next
A	9	C
B	16	C
C	4	-
D	12	C

two node instability (or) Count to infinity.

to Avoid this:

* split horizon

* poisson reverse

RIP messages

* request

* response

if (requested)
h₂ response

Solicited

Unsolicited

if (!requested)

h₂ periodically

response

has routing
table information

RIP timer:

only one timer

* periodic timer:

Advertising

regular

update
messages

25 - 30 sec

* Expiration timer:

* It governs the validity of a
route. (180 sec)

* the route is considered expired
and the hop count of the
route is set to 16.

* Once we get any update
for any route, the expiration
timer resets.

Garbage Collection timer:

An invalid route is not purged from the routing table until the timer expires (120 sec)

Once a route is expired, this GC timer starts to tick.

hence a route will be 300 sec in the table.

20 entries in table no update

For 200s for 5 routes. how many timers are running?

1 periodic timer

15 expiration timer

5 Garbage Collection timer

total: $n+1$ timers will be running in

total n is no. of routes

Multiple-access protocol

* multiple users are trying to access

multiple-access protocol

random access protocol

↳ ALOHA

if collision is detected

↳ CSMA/CD

↳ CSMA/CA

change (Random Back off time)

wireless medium

(Send RTS, CTS)

ready to send

clear to send

Random Access:

* Contention (who is going to send) based protocol

ALOHA → pure ALOHA
 → slotted ALOHA

Controlled access protocol

↳ reservation

↳ polling

↳ token passing

Channelization protocols

↳ FDMA

↳ TDMA

↳ CDMA

Pure ALOHA:

* I don't consider any collision, I will keep on sending the data if I have any data, no bother about medium.

Slotted ALOHA:

* the time will be slotting the time.

* If a slot begin we need to send data immediately and complete by the end of the slot.

* We can't send during middle of the slots.

CSMA:

* Carrier sense multiple Access.

* Carrier is the medium.

"Sense before transmit"

or

"listen before talk"

* If medium is free we will transmit else we will wait.

types:

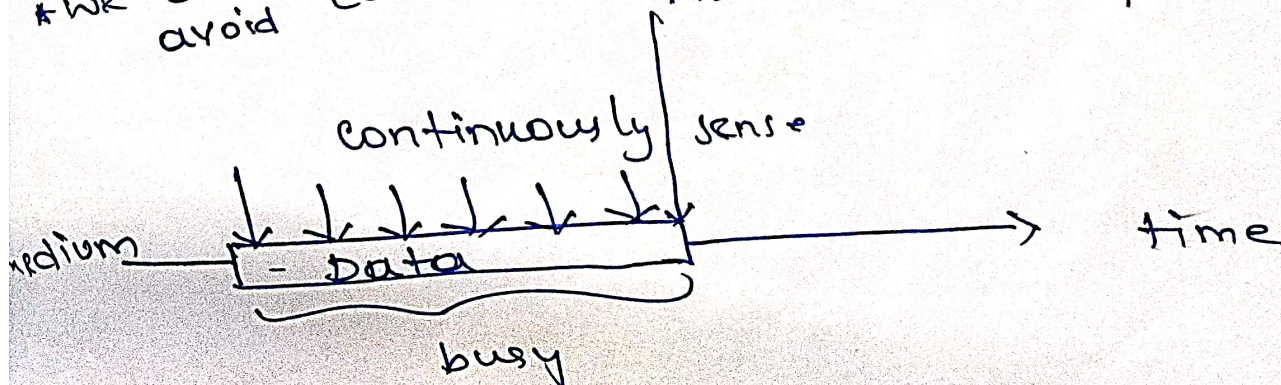
* 1- persistent

* non-persistent.

1-persistent:

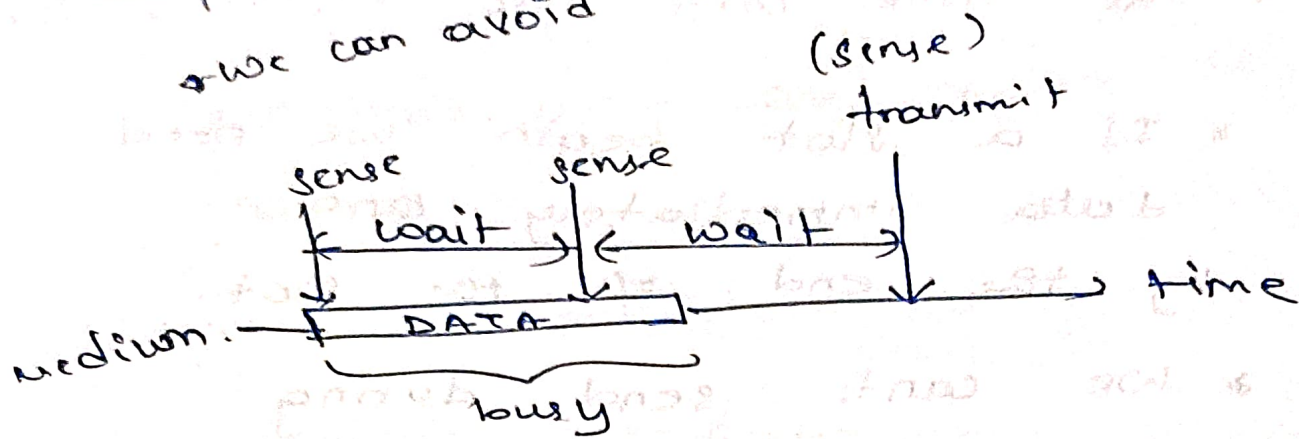
* We can't avoid collision

Transmit (with probability 1)



Non-persistent:

we can avoid collision.



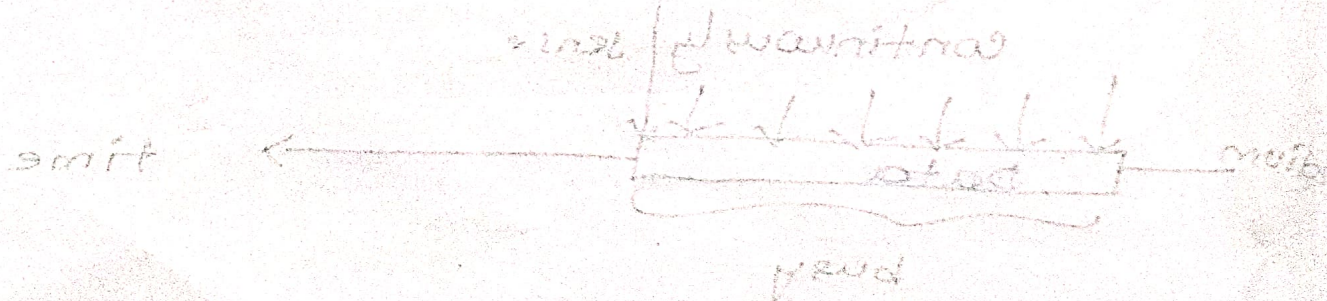
the waiting time is random and not same [Random Back off time]

If medium is free we will

transmit else we will wait

transmit (with probability 1)

we can avoid collision



Process-to-Process Delivery

UDP, TCP

node-to-node : Data link layer

host-to-host : Network layer

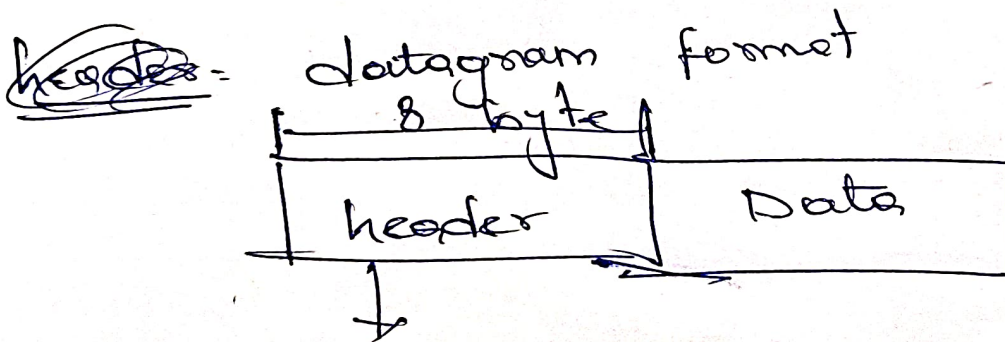
process-to-process : Transport layer.

* the process are found & identified by logical port number.

Socket address = IP + port number
logical port number

UDP :

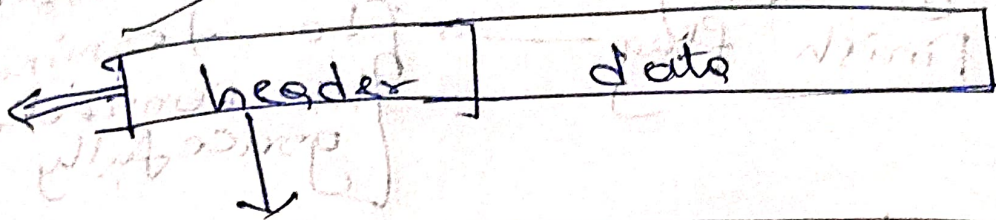
~~is~~ * connectionless protocol



Source port number (16 bits)	Destination port number (16 bits)
Total length (16 bits)	Checksum (16 bits)

• Connection oriented protocol

segment format segment:



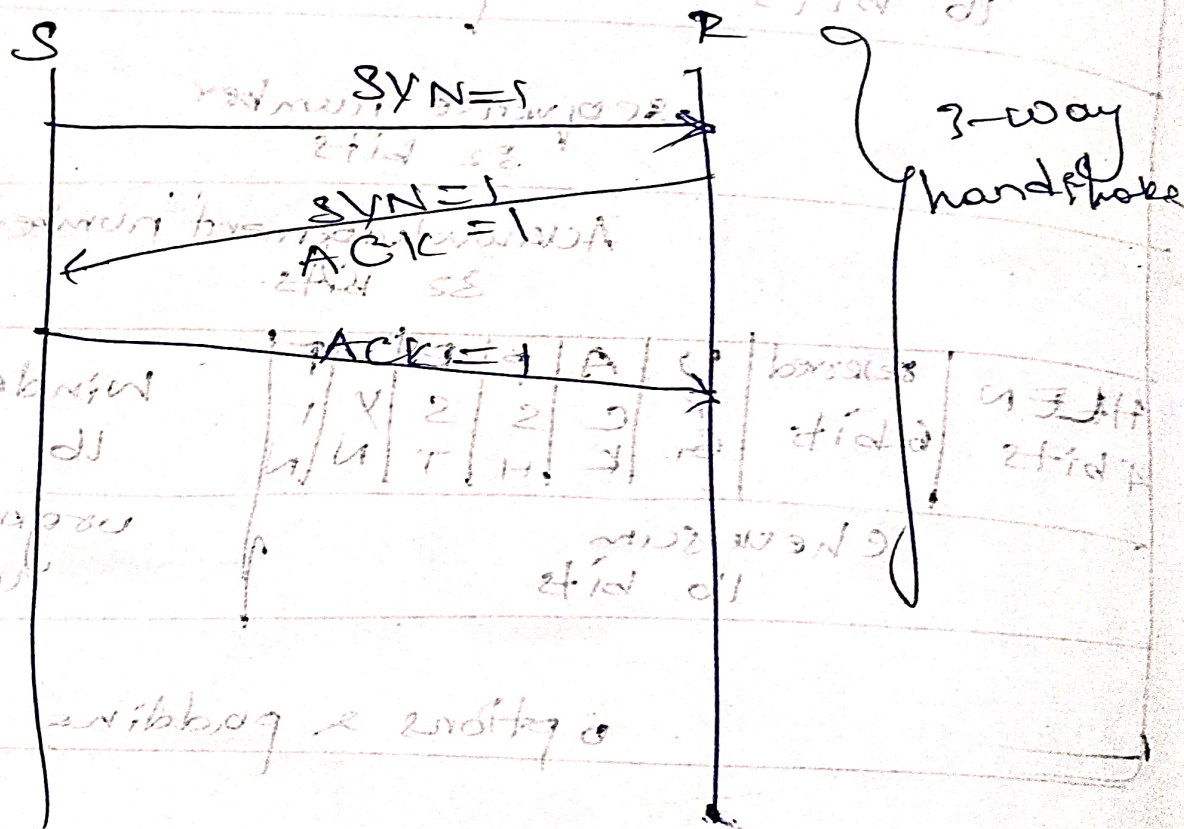
Source port address 16 bits		Destination port address 16 bits					
Sequence number 32 bits							
Acknowledgement number 32 bits							
HLEN 4 bits	Reserved 6 bits	U	A	P	R	S	F
		R	C	S	S	Y	I
		G	K	H	T	N	N
Checksum 16 bits		Window size 16 bits					
		Urgent pointer 16 bits					
Options & padding							

Acknowledgement number = sequence number + 1

Acknowledgement

- URG - Urgent flag
- ACK - Acknowledgement flag
- PSH - Push flag → once we receive data we will push to data other
- RST - reset flag → abruptly disconnected
- SYN - Sync flag → to establish the connection
- FIN - Finish flag → to terminate the connection gracefully

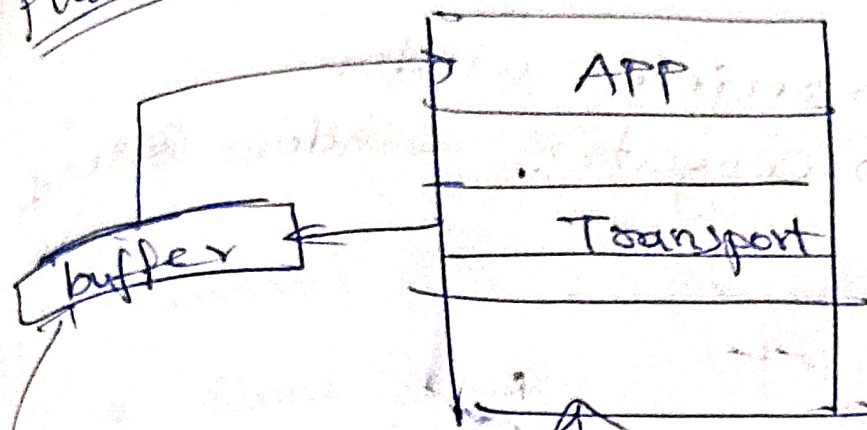
While establishing connection, the first message is sent by sync flag set



Active open = whoever initiates connection is active open

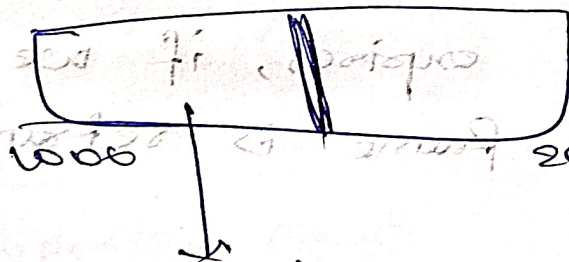
Passive open = whoever listens is called passive open.

Push Flag:



push flag is set to push the data (whatever in buffer) to upper / lower layers even though the buffer is ~~not~~ not full

Urgent flag:



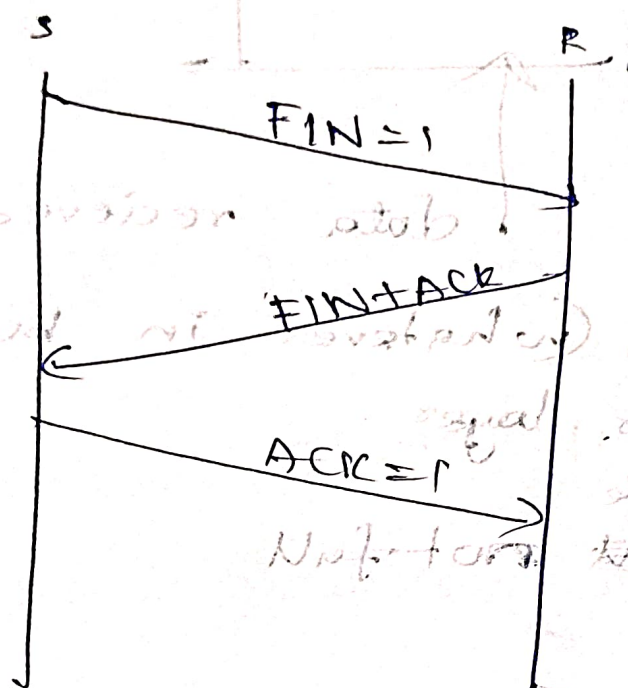
immediate processing is needed (1000 - 500) just send these data and set the urgent flag & the starting offset is at urgent pointer)

Window Size = buffer size

rwnd → receiver window

cwnd → Congestion window @ sender

Termination:



Fast-retransmission:

before RTO expires, if we send 3 ACK the lost frame is retransmitted.

if window is too small (e.g. 1000 - 1000) then we must have a larger window (e.g. 10000 - 10000) to avoid the window being too small and causing a timeout (e.g. 10000 - 10000) to occur.

data rate > bandwidth.

Control

TCP Congestion

* Slow start, exponential increase
↳ window size increases exponentially (2^n , $n=0,1,2,\dots$)

* Additive increase, Multiplicative decrease

↳ window size increase by 1.

Tahoe

TCP Tahoe = Slow start + AIMD + fast transmission.

Example:

Suppose $cwnd = 200$ & packet loss occurs, then

$ssthresh = cwnd/2$ & $cwnd$ is set to 1

now slow start with $cwnd$ as

til $ssthresh = 100$

after $ssthresh > 100 \Rightarrow$ start AIMD

if there is a loss in AIMD ie say 125,

ss thresh = $125/2 = 62$ & cwnd = 1

& again starts with slow start.

& the steps increase by RTT factor

1 RTT = Round Trip Time

TCP Reno

TCP Reno = TCP Tahoe + Fast recovery

* if Congestion is detected by 3-ACK's

cwnd = $cwnd/2$ & ssthresh = $cwnd/2$

& skips slow start & starts

with AIMD.

* if Congestion is detected by

RTO (Timer) cwnd = 1 & ssthresh = $\frac{cwnd}{2}$

& starts from slow start following

slow start, AIMD will begin.